

Оператори за цикъл. Разработване на програми с циклична структура.

I. Теоретична постановка

1. Цикъл for

Този цикъл позволява определен брой елементи да бъдат обходени последователно. Форматът на цикъл for е:

```
for <елемент> in <последователност>
    <тяло на цикъла>
[else:
    <блок, който ще бъде изпълнен, ако не бъде използван оператор break>
]
```

Частта, оградена с квадратни скоби не е задължителна. Елемент е променлива, чрез която ще бъде достъпен текущия елемент при обхождане. Последователност е поредицата от данни, която се обхожда. Това може да е низ, списък, кортеж, речник и т.н. Тялото на цикъла съдържа операциите, които ще бъдат изпълнени при всяка итерация на цикъла.

Цикълът for може да се използва и като броячен цикъл, т.е. цикъл, в който се изрежда определена последователност от стойности. За целта той трябва да се комбинира с функция range().

Форматът на функция range() е следния:

```
range([начало,] край [, стъпка])
```

Частите, оградени с квадратни скоби не са задължителни. Ако функцията има само един параметър, той се интерпретира като край на изброяваните стойности. Тогава по подразбиране за начало на поредицата се използва стойност 0 т.е. range(10) е равносилно на range(0,10). Стойността на параметъра край не се включва в създаваната последователност, т.е. ако бъде извикана функцията range(0,10), получената последователност ще включва числата от 0 до 9. Параметърът стъпка задава увеличението, с което се променят последователните стойности в поредицата. По подразбиране той има стойност 1. Стъпката може да бъде и отрицателна. В този случай стойността на числата от поредицата ще намалява.

2. Цикъл while

Този цикъл се изпълнява дотогава, докато логическото условие е вярно.

Форматът на цикъл while е:

```
while <логически израз>:
    <тяло на цикъла>
[else:
    <блок, който ще бъде изпълнен, ако не бъде използван оператор break>
]
```

Частта, оградена с квадратни скоби не е задължителна. Операторът while трябва да се използва внимателно. Ако в тялото на цикъла не е осигурено изменение на логическия израз, ще се получи безкраен цикъл и т. нар. „зацикляне“ на програмата. Безкраен цикъл може да бъде прекратен посредством клавишна комбинация Ctrl+C, но това ще спре и цялостното изпълнение на програмата. При цикъл for не може да се получи безкраен цикъл, защото няма безкрайни поредици от данни.

Преднамерен безкраен цикъл може да се постигне по следния начин:

```
while True:
    блок_от_код
```

3. Оператори break и continue

Оператор break прекратява изпълнението на цикъл предсрочно, а оператор continue прекъсва текущата итерация на цикъла и осъществява преход към следващата.

Пример 1: Да се напише програма, която намира сумата от 10 въведените от потребителя числа с цикъл for.

```
sum=0          # Нулиране на сумата
for i in range(0,10): # Препробяване до 10
    a=int(input("a = "))
    sum+=a      # Натрупване на числата
print(f"sum = {sum}") # Друг вариант за форматиране е      print("sum = %d" % sum)
```

Пример 2: Да се напише програма, която намира сумата от 10 въведените от потребителя числа с цикъл while.

```
sum=0          # Нулиране на сумата
index=0        # Нулиране на брояча на числата
while index<10:
    a=int(input("a = "))
    sum+=a      # Натрупване на числата
    index+=1    # Промяна на броя на числата !!!
print(f"sum = {sum}") # Друг вариант за форматиране е      print("sum = %d" % sum)
```

Обърнете внимание, че в този случай стъпката не се променя автоматично и трябва ръчно да се променя броя на числата, включвани в сумата. Ако пропуснете тази стъпка, ще влезете в безкраен цикъл.

Пример 3: Напишете програма, която намира сумата само от положителните числа в поредица от въведени от потребителя стойности. Използвайте стойност -99 за край на въвеждането.

```
sum=0          # Нулиране на сумата
while True:    # Стартиране на безкраен цикъл
    a=int(input("a = ")) # Въвеждане на число
    if a== -99:         # Проверка за край на въвеждането
```

```
break          # Прекъсване на цикъла
if a<0:         # Проверка за отрицателни числа
    continue    # Преминване към следващата итерация
sum+=a         # Сумиране на положителните числа
print(f"sum = {sum}") # Друг вариант за форматиране е      print(f"sum = %d" % sum)
```

Помислете какво ще се случи ако разменим местата на двете проверки. Ще продължи ли програмата да работи правилно?

II. Задачи за самостоятелна работа

1. Направете промени в кода на Пример 1, така че потребителят сам да определя броя на числата включени в сумата. Добавете проверка, която да определя дали потребителят е въвел положително различно от 0 число за брой на числата в сумата.

Забележка: Добавете още една променлива limit, в която потребителят сам да въвежда броя на числата в поредицата.

2. Направете промени в кода на Пример 2, така че в нея да се определя дали получената сума е четно или нечетно число.

Забележка: Добавете проверка за четност чрез деление по модул 2 на изчислената сума.

3. Направете промени в кода на Пример 3, така че в нея да се определя и броя на въведените положителни числа.

Забележка: Добавете още една променлива br, с която да се преброяват положителните числа. Не забравяйте да я нулирате преди да я използвате. Не забравяйте, че 0 не е нито положително, нито отрицателно число и не трябва да се включва в този брой.

4. Напишете програма на Python, която десет цели числа на избрана степен. Всички числа и техните степени се въвеждат от потребителя.

Забележка: Операторът за повдигане на степен в Python е **. Отляво на оператора е числото, а отдясно е степента.

Помислете как трябва да се промени програмата, така че броят на повдиганите на степен числа да се въвежда от потребителя.